

Assignment 1: Observability

Inora Observability Project

Enterprise Software Development, Fall 2026

Prepared by Maaz Ahmed Siddiqui

September 2026

Part A: The Project

The Problem

Inora is a jewelry store. When a customer tries to buy a piece, one of three things can happen. The order can succeed, the customer can cancel partway through, or the backend can fail quietly, for example because of a payment error. From the outside, a cancelled order and a crashed order look the same. Both are just a checkout that did not finish. Without a way to tell these apart, Inora's team cannot know if people are leaving on purpose or if something is actually broken and costing them sales. The team also has no way to know the site is running slow until customers start complaining, and by that point sales are already lost.

Intended Users

This project is built for Inora's small development and operations team. These are the people who need a fast answer to two questions. Is checkout working right now, and if an order failed, was that a customer choosing to cancel or was it a real system error.

The Solution

This project adds an observability layer on top of a Flask backend that simulates Inora's storefront. It has two main parts.

- A live Grafana dashboard that separates business outcomes, meaning orders placed, cancelled, or failed, from technical health, meaning request speed, error counts, and server load. The team can look at one screen and tell whether a rise in incomplete orders is customers cancelling or the backend failing.
- A logging pipeline built with Filebeat, Elasticsearch, and Kibana. Every request gets a unique `request_id`. If a checkout fails, the team can search that exact id in Kibana and see exactly what happened, instead of guessing.

What Works

- GET /catalog: lists jewelry items, with a small random delay to act like real response time variation.
- POST /order: places an order. About 20 percent of attempts fail on purpose, to act like a real payment failure.
- POST /cancel: a genuine cancellation action, tracked separately from a failed order.
- Every request is written to a log file as one JSON line, with a time, a severity, a message, and a `request_id`.
- Every request is measured by Prometheus, covering request counts, requests in progress, response time, and order processing time. All of this is shown on a Grafana dashboard, along with the CPU, memory, disk, and network use of the machine running the project.

How to Try It

Full setup steps are in the project's README file. In short:

- Start the stack with: `docker compose up -d`
- Set up Python: `python -m venv venv`, then activate it, then `pip install -r requirements.txt`
- Run the app: `python app.py`
- Run the traffic generator in a second terminal: `python traffic.py`
- Open Grafana at `localhost:3000` and Kibana at `localhost:5601` to watch the data come in.

Part B: Metrics

A Prometheus client library was added to the Flask backend, and a `/metrics` endpoint was created so Prometheus can read the current numbers every few seconds. Five metrics were built directly for this project, covering all four metric types the assignment asks for, plus four more metrics that come from Node Exporter and describe the health of the machine itself.

Metrics Table

Metric Name	Purpose	Type	Unit	Labels	Code Location
app_requests_total	Counts every request the app receives, split by which endpoint, method, and whether it succeeded or failed	Counter	requests (count)	method, endpoint, http_status	app.py, inside <code>catalog()</code> and <code>order()</code>
business_orders_total	Counts what actually happened to an order: it succeeded, it failed, or the customer cancelled it	Counter	orders (count)	status (success, failed, cancelled)	app.py, inside <code>order()</code> and <code>cancel()</code>
app_active_requests	Shows how many requests are being handled right now, at this exact moment	Gauge	requests (in progress)	none	app.py, the <code>ACTIVE_REQUESTS.track_inprogress()</code> decorator on <code>catalog()</code> and <code>order()</code>
app_request_latency_seconds	Measures how long the <code>/catalog</code> endpoint takes to respond, bucketed so percentiles can be worked out	Histogram	seconds	none (1e buckets are added automatically)	app.py, the <code>REQUEST_LATENCY.time()</code> decorator on <code>catalog()</code>
app_order_processing_seconds	Measures how long it takes to process a successful order	Summary	seconds	none	app.py, <code>ORDER_PROCESSING.observe()</code> inside <code>order()</code>

Why a Histogram for one and a Summary for the other

A Histogram was used for catalog latency because the manual points out that Prometheus client libraries in Python cannot calculate percentiles inside a Summary metric. A Summary can only report a fixed set of quantiles that get calculated on the client side, and this project's client library does not support that. A Histogram sends bucketed counts to Prometheus instead, and Prometheus itself works out the percentile using the `histogram_quantile` function. This is why the p95 chart in this project is built from the Histogram, not the Summary. The Summary metric is still useful on its own, since it gives a simple average without needing any extra math.

The Code Behind the Metrics

```
REQUEST_COUNT = Counter('app_requests_total', 'Total requests', ['method', 'endpoint', 'http_status'])
ACTIVE_REQUESTS = Gauge('app_active_requests', 'Active requests')
REQUEST_LATENCY = Histogram('app_request_latency_seconds', 'Request latency')
ORDER_PROCESSING = Summary('app_order_processing_seconds', 'Time spent processing orders')
BUSINESS_ORDERS = Counter('business_orders_total', 'Total orders placed', ['status'])

@app.route('/catalog')
@ACTIVE_REQUESTS.track_inprogress()
@REQUEST_LATENCY.time()
def catalog():
    time.sleep(random.uniform(0.1, 0.3))
```

```
REQUEST_COUNT.labels('GET', '/catalog', '200').inc()
...
```

The Grafana Dashboard, Panel by Panel

Total Orders Placed

Query: `sum(business_orders_total) by (status)`

This is the panel built specifically for this project, beyond what was covered in class. It plots success, failed, and cancelled as three separate lines using the same underlying counter, so the team can see at a glance whether a drop in completed orders is customers cancelling on purpose or the backend actually failing. This is the panel that answers the exact business question in Part A.

Active Requests

Query: `app_active_requests`

Shows how many requests are being handled at once. It should sit at 0 when nothing is happening and rise briefly while a request is in progress.

p95 Request Latency (5m)

Query: `histogram_quantile(0.95, sum(rate(app_request_latency_seconds_bucket[5m])) by (le))`

Shows the value under which 95 percent of catalog requests finish, calculated over a rolling 5 minute window. This is explained in full detail below.

Average Order Processing Time

Query: `rate(app_order_processing_seconds_sum[5m]) / rate(app_order_processing_seconds_count[5m])`

Divides the total time spent processing orders by the number of orders processed, over the last 5 minutes, to give a simple rolling average.

CPU Usage (%)

Query: `100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`

Shows how busy the machine's processor is, worked out as 100 minus the percentage of time it spent idle.

Memory Usage (%)

Query: `100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)`

Shows what percentage of the machine's memory is currently in use.

Disk Usage (%)

Query: `100 - ((node_filesystem_avail_bytes{mountpoint="/mnt/host/c"} * 100) / node_filesystem_size_bytes{mountpoint="/mnt/host/c"})`

Shows what percentage of the machine's C: drive is in use. Node Exporter runs inside a container on Docker Desktop, and Docker exposes the real Windows C: drive into that container at the path `/mnt/host/c`, so this reading reflects the actual host disk, not a virtual one.

Network I/O

Query: `rate(node_network_receive_bytes_total{device!="lo"}[5m]) and rate(node_network_transmit_bytes_total{device!="lo"}[5m])`

Plots incoming and outgoing network traffic as two lines. This machine only has one real network device, called eth0 inside the Docker Desktop Linux environment, and the loopback device lo is excluded since it is not real network traffic.

Understanding the p95 Time Window

The p95 chart uses a 5 minute window, written as [5m] in the query. This means that at any moment, the chart is looking back over the last 5 minutes of traffic and working out the value under which 95 percent of requests in that window finished. It is a rolling window, so it constantly slides forward. This matters a lot for reading the chart correctly. If something changes, like a delay being added to the code, the chart does not jump immediately. For the first few minutes, the window still contains a mix of old, normal requests and new, slow requests, so the number only partly reflects the change. It takes close to the full 5 minutes for the window to be made up entirely of the new behaviour. This exact effect was seen directly during the experiment in Part E, and is explained there with real numbers.

Node Exporter and the Machine Being Measured

Node Exporter was added to the Docker setup to report on the health of the machine the project runs on, separate from anything about the Flask app itself. Prometheus scrapes it every 5 seconds, the same interval used for the app. Every metric coming from Node Exporter carries the label machine="DESKTOP-682RE3R", which is the actual hostname of the machine this project was built and tested on, found by running the hostname command. This makes it clear, if this project were ever run on more than one machine, exactly which machine each reading belongs to.

One honest note worth including here. Node Exporter runs inside a Docker container, and on Windows, Docker Desktop actually runs containers inside a small Linux environment called WSL2, not directly on Windows. This means CPU and memory readings reflect that Linux environment rather than Windows' own task manager numbers, though they still track the same real physical machine underneath. The disk reading is the exception, since Docker Desktop passes the real Windows C: drive through to the container directly, so that number is a true reading of the actual disk.



Figure: CPU, memory, disk, and network panels, all four Node Exporter metric types, shown together on the dashboard.

Part C: Logs

What Is Logged, Why, and Where

Every meaningful action in the app writes one line to a log file, in JSON format. This includes a customer viewing the catalog, an order succeeding, an order failing on the backend, and a customer cancelling an order. Each line carries five fields: the time it happened, a fixed `service_name` identifying which service wrote it, a severity level such as INFO or ERROR, a plain message describing what happened, and a `request_id` that is unique to that one request. The `request_id` is the most important part for this project, since it lets the team search Kibana for one exact event and trace it, instead of scrolling through a wall of text.

The field was named `service_name` instead of the more obvious `service` on purpose. Elasticsearch reserves the field name `service` for its own internal schema, and using it directly caused every log line to be rejected with an indexing error. Renaming the field to `service_name` fixed this completely.

```
class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "time": self.formatTime(record),
            "service_name": "inora-backend",
            "severity": record.levelname,
            "message": record.getMessage(),
            "request_id": getattr(record, "request_id", "n/a")
        })

handler = logging.FileHandler("app.log")
handler.setFormatter(JsonFormatter())
logging.getLogger().addHandler(handler)
```

How Filebeat Collects and Parses the Logs

Filebeat runs as its own container and watches the `app.log` file through a read only volume mount. It reads the file line by line using a `filestream` input, and because each line is written as one complete JSON object, Filebeat's built in `ndjson` parser can decode it directly. The setting `target: ""` tells Filebeat to place every decoded field, such as `severity` and `request_id`, directly at the top level of the document it sends to Elasticsearch, rather than nesting them inside a sub object. This is what makes `severity` and `request_id` show up as their own separate, searchable columns in Kibana, instead of being buried inside one long message field.

```
filebeat.inputs:
- type: filestream
  id: inora-app-logs
  enabled: true
  paths:
  - /usr/share/filebeat/app.log
  parsers:
  - ndjson:
    target: ""
    add_error_key: true

output.elasticsearch:
  hosts: ["elasticsearch:9200"]
```

Where Logs Live, and How Long They Last

Logs are stored inside the Elasticsearch container, in a Docker managed storage area. The `docker-compose.yml` file for this project does not define a named, persistent volume for Elasticsearch, which has two direct effects. First, log data does survive a normal restart of the containers, and it survives the computer itself restarting, since Docker keeps this storage around by default. Second, if the containers are ever removed with the command `docker compose down -v`, that storage is deleted completely and every log is gone for good. This command was actually used once during this project, deliberately, to get a clean, empty start before collecting the final results for this report.

There is no automatic deletion of old logs configured in this project. Nothing ages out on its own, and log data would simply keep growing forever if the project ran for a long time. In a real production setup, this would normally be solved with a feature in Elasticsearch called Index Lifecycle Management, which can be configured to automatically delete log data past a certain age, for example anything older than 30 days. Setting this up was outside the scope of what this assignment needed, since the project only runs for short, local testing sessions, but it is worth naming as the correct next step for a real deployment.

Searching in Kibana

To find a specific event, the Discover tab in Kibana is used with the KQL search bar. Searching severity: "ERROR" shows every error across the whole app. To trace one specific request from start to finish, its exact request_id is searched instead, for example request_id: "1e789a30", which returns exactly the one log line that request produced. A real example of both is shown below.

Sample Stored Log

```
{
  "time": "2026-09-23 10:16:37,725",
  "service_name": "inora-backend",
  "severity": "ERROR",
  "message": "Payment transaction failed for order attempt",
  "request_id": "1e789a30"
}
```

message	Payment transaction failed for order attempt
request_id	1e789a30
service_name	inora-backend
severity	ERROR
time	2026-09-23 10:16:37,725

Figure: the same log line, opened in Kibana, showing message, request_id, service_name, severity, and time as separate fields.

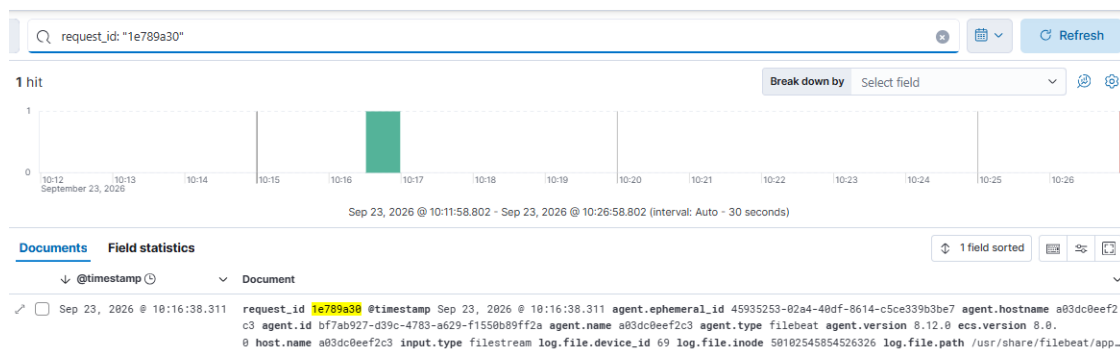


Figure: searching request_id: "1e789a30" in Kibana returns exactly one hit, the same log line shown above.

Part D: System Design

Architecture Diagram

The diagram below shows every part of the project and how they connect. Arrows show the direction data moves in, and short notes explain what happens if a piece goes down, where data is actually stored, and why.

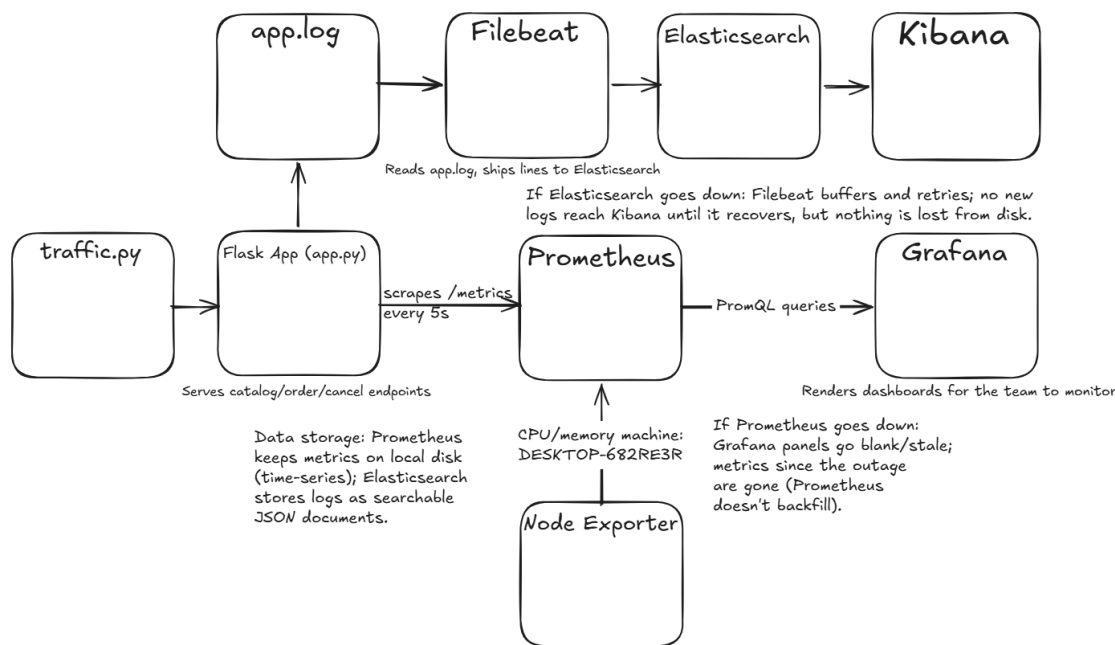


Figure: full architecture diagram, covering the app, the metrics pipeline, and the logging pipeline.

What Each Part Actually Does

Component	What it does
traffic.py	A small script that continuously calls the app's endpoints, so there is always live data flowing through the system to observe.
Flask App (app.py)	Serves the catalog, order, and cancel endpoints. This is the actual application being observed.
Prometheus	Every 5 seconds, reaches out to the app's /metrics endpoint and to Node Exporter, and saves whatever numbers it finds, with a timestamp, into its own local storage.
Grafana	Runs the PromQL queries shown in Part B against Prometheus, whenever a dashboard is open or refreshed, and draws the results as charts.
Node Exporter	Reads CPU, memory, disk, and network statistics directly from the host machine and makes them available for Prometheus to read.
Filebeat	Watches the app.log file line by line, decodes each line as JSON, and forwards the decoded fields to Elasticsearch.
Elasticsearch	Stores and indexes every log line it receives from Filebeat, so that it can be searched quickly by any field.
Kibana	Provides the search and browsing interface on top of Elasticsearch. This is where a request_id gets searched to trace one specific event.

Where Data Is Stored, and Why

- Prometheus keeps metrics on its own local disk, as a time series, meaning a value paired with a timestamp. It is treated as disposable in this project. If it were lost, it would simply refill itself from the next round of scrapes, so nothing important depends on it long term.
- Elasticsearch stores logs as searchable JSON documents. This is treated as more important than Prometheus's data, since a lost log line about a real payment failure cannot be recreated later the way a metric can.
- The original copy of every log line still exists as plain text in app.log on disk, even after Elasticsearch has it too. Elasticsearch holds a searchable copy, not the only copy.

What Happens If a Component Stops Working

Component	Impact if it goes down
Flask App	The app itself stops responding to real requests. This is the only failure that would actually be visible to a customer trying to use the store.
Prometheus	No new metrics get collected while it is down, so every Grafana panel goes flat or blank for that period. The app keeps working fine for customers, since Prometheus only reads data, it does not affect the app.
Grafana	The dashboards become impossible to view, but Prometheus keeps collecting and storing data underneath. Nothing is lost, it just cannot be seen until Grafana is back.
Node Exporter	CPU, memory, disk, and network panels stop updating. The app's own metrics, like order counts and latency, are completely unaffected, since they come from the app directly, not from Node Exporter.
Filebeat	New log lines keep being written safely to app.log on disk. Nothing reaches Elasticsearch until Filebeat comes back, and then it picks up again from where it left off.
Elasticsearch	Filebeat's attempts to send data fail and it keeps retrying. Kibana stops working since it has nothing to search. The app's own writing to app.log is not affected at all.
Kibana	Searching logs becomes unavailable, but the data sitting inside Elasticsearch is completely safe and untouched. The app itself is unaffected.

Following One Metric, Start to Finish

The metric chosen here is `app_request_latency_seconds`, the Histogram behind the p95 Request Latency panel.

Step 1: The code records it

In `app.py`, the `/catalog` route is wrapped with the decorator `@REQUEST_LATENCY.time()`. This starts a timer the moment a request to `/catalog` begins, and stops it the moment the function returns, automatically adding that duration into the Histogram's buckets.

```
@app.route('/catalog')
@ACTIVE_REQUESTS.track_inprogress()
@REQUEST_LATENCY.time()
def catalog():
    time.sleep(random.uniform(0.1, 0.3))
    ...
```

One real observation from watching this project run is worth explaining honestly. The baseline p95 reading turned out to be about 0.46 seconds, even though the code's own random delay only ever goes up to 0.3 seconds. This is not a bug. Flask's built in development server, which this project uses, handles requests one at a time by default. While `traffic.py` is firing `catalog` and `order` requests close together, a `catalog` request can end up waiting for an earlier `order` request to finish first, since the server can only work on one at a time. That waiting time gets counted as part of the `catalog` request's total duration, which is why the real, observed number came out a bit higher than the code's own delay range on paper. This is a genuine and common limitation of a development server, not something wrong with the metric itself.

Step 2: Prometheus collects and stores it

Every 5 seconds, exactly as set in `prometheus.yml`, Prometheus sends a request to `host.docker.internal:5000/metrics` and reads the current bucket counts as plain text. Each scrape is saved with its own timestamp, so a full history builds up over time rather than each new reading replacing the old one.

```
- job_name: 'inora_backend'
  static_configs:
    - targets: ['host.docker.internal:5000']
```

Step 3: Grafana queries and displays it

The p95 Request Latency panel runs this query every time it loads or refreshes:

```
histogram_quantile(0.95, sum(rate(app_request_latency_seconds_bucket[5m])) by (le))
```

This turns the raw bucket counts into a rate over the last 5 minutes, and then works out the value under which 95 percent of requests fall. The real values recorded for this project were 0.4616 seconds under normal conditions, and 0.6875 seconds once a deliberate delay was added, both shown in full detail in Part E below.

Following One Log, Start to Finish

The log chosen here is a real ERROR entry produced when a simulated payment failed, carrying the request_id 1e789a30.

Step 1: The code writes it

```
if random.random() < 0.2:
    logging.error("Payment transaction failed for order attempt", extra={"request_id":
req_id})
    return {"error": "payment_failed"}, 500
```

This produces one JSON line in app.log, shown earlier in Part C, with severity set to ERROR and the request_id attached.

Step 2: Filebeat picks it up and parses it

Filebeat notices the new line in app.log within a second or two, decodes it as JSON using its ndjson parser, and sends the decoded fields to Elasticsearch.

Step 3: Elasticsearch stores it

The entry is stored as a searchable document, with its own fields plus a few extra fields Elasticsearch adds automatically, such as an @timestamp used for Kibana's time filter.

Step 4: Kibana finds it

Searching request_id: "1e789a30" in Kibana's Discover tab returns exactly one hit, the same entry, with every field visible separately. This was shown earlier in Part C with real screenshots.

Part E: Experiments

Experiment 1: Reproducing a Slowdown

Problem chosen: a delay added to some catalog requests, to act like a real slow dependency, for example a slow database or a slow third party service.

Normal Behaviour

With app.py and traffic.py both running normally, the p95 Request Latency panel was left to run for 5 minutes. The value read directly from Prometheus at this point was 0.4616 seconds. This is the baseline.



Figure: dashboard under normal conditions. p95 latency sits around 0.46 seconds.

Prediction

Before the delay was added, the expected outcome was written down first. Adding a 500 millisecond delay to 1 in every 5 catalog requests should push the p95 Request Latency panel noticeably higher, since the 95th percentile line sits close to the edge of that affected group. No error logs were expected to appear, since this delay only makes a request slower, it does not make it fail. It was also expected that the change would not show up immediately, because of the 5 minute rolling window explained in Part B, and that the full effect would only be visible once enough time had passed for the window to no longer contain older, normal traffic.

Introducing the Problem

```
if random.random() < 0.2:      # 1 in 5 requests
    time.sleep(0.5)             # add 500ms delay
```

This was added directly after the existing random delay inside the /catalog route, and app.py was restarted. traffic.py kept running without any change.

What Actually Changed

At 3 minutes after the restart, the p95 reading was 0.475 seconds, only a small rise from the 0.4616 second baseline. This matched the prediction that the window would still be diluted with older, pre fault traffic. At 5 or more minutes after the restart, once the entire 5 minute window was made up of traffic from during the fault, the p95 reading rose clearly to 0.6875 seconds.



Figure: dashboard during the fault. p95 latency has visibly risen, and Total Orders Placed and other panels show a clear spike at the same point in time.



Figure: CPU, memory, disk, and network also rise briefly during the fault, since the app is doing more work per request.

Kibana logs during this window were also checked, to confirm the fault was only affecting speed, not success. Every log line found during this period was still severity INFO, with the normal "Catalog endpoint accessed by user" message, confirming that requests were completing successfully, just more slowly.

```
✓ [ ] Sep 23, 2026 @ 10:47:51.712 message Catalog endpoint accessed by user @timestamp Sep 23, 2026 @ 10:47:51.712 agent.ephemeral_id 45935253-02a4-48df-8614-c5ce339b3be
7 agent.hostname a03dc0eef2c3 agent.id bf7ab927-d39c-4783-a629-f1550b89ff2a agent.name a03dc0eef2c3 agent.type filebeat agent.version 8.1
2.0 ecs.version 8.0.0 host.name a03dc0eef2c3 input.type filestream log.file.device_id 69 log.file.inode 50102545854526326 log.file.path /_

✓ [ ] Sep 23, 2026 @ 10:47:51.710 message Catalog endpoint accessed by user @timestamp Sep 23, 2026 @ 10:47:51.710 agent.ephemeral_id 45935253-02a4-48df-8614-c5ce339b3be
7 agent.hostname a03dc0eef2c3 agent.id bf7ab927-d39c-4783-a629-f1550b89ff2a agent.name a03dc0eef2c3 agent.type filebeat agent.version 8.1
2.0 ecs.version 8.0.0 host.name a03dc0eef2c3 input.type filestream log.file.device_id 69 log.file.inode 50102545854526326 log.file.path /_

✓ [ ] Sep 23, 2026 @ 10:47:49.785 message Catalog endpoint accessed by user @timestamp Sep 23, 2026 @ 10:47:49.785 agent.ephemeral_id 45935253-02a4-48df-8614-c5ce339b3be
7 agent.hostname a03dc0eef2c3 agent.id bf7ab927-d39c-4783-a629-f1550b89ff2a agent.name a03dc0eef2c3 agent.type filebeat agent.version 8.1
2.0 ecs.version 8.0.0 host.name a03dc0eef2c3 input.type filestream log.file.device_id 69 log.file.inode 50102545854526326 log.file.path /_
```

Figure: catalog log lines during the fault window, all still severity INFO, confirming no failures were caused by the delay.

Cause and Effect

This experiment models what would happen if a real dependency behind the catalog page, such as a database or an external service, became slow. From a customer's point of view, the page would take noticeably longer to load some of the time, though it would never actually fail or show an error. Since no errors are produced, a monitoring setup that only watches for failed requests would completely miss this problem. This is exactly why a latency metric like this one matters on its own, separately from error tracking.

Recovery

The added delay code was removed, and app.py was restarted again. traffic.py kept running the whole time. After 5 minutes, once the rolling window was made up entirely of post fault traffic, the p95 reading was checked again and had returned to 0.4532 seconds, close to the original 0.4616 second baseline, with the small difference explained by normal variation in traffic.



Figure: dashboard after removing the fault. p95 latency has dropped back down to normal.



Figure: CPU, memory, disk, and network also return to their normal resting levels after recovery.

Summary of Results

Stage	p95 Request Latency
Baseline	0.4616 seconds
During fault, 3 minutes in (window still diluted)	0.475 seconds
During fault, 5+ minutes in (window fully inside fault)	0.6875 seconds
After recovery, 5 minutes later	0.4532 seconds

Experiment 2: Cardinality Explosion

A cardinality explosion happens when a metric label is given too many different possible values, for example a unique id. Prometheus creates a completely separate time series for every different value a label can take, so a label with thousands of different values creates thousands of separate series from what was meant to be one single metric.

The Experiment

A small, separate test script was written, completely outside the main app, so the real project metrics would not be affected.

```
from prometheus_client import Counter, start_http_server
import uuid, time
```

```
demo_requests = Counter('demo_requests_total', 'test counter', ['request_id'])
start_http_server(8001)

for i in range(100):
    demo_requests.labels(request_id=str(uuid.uuid4())).inc()
    time.sleep(1)
```

This counter is given a `request_id` label, and the script generates 100 different random ids, incrementing the counter once for each one.

Series Growth

While the script was running, the query `count(demo_requests_total)` was run several times in Prometheus's own web interface, to watch the number climb rather than only checking it once at the end.

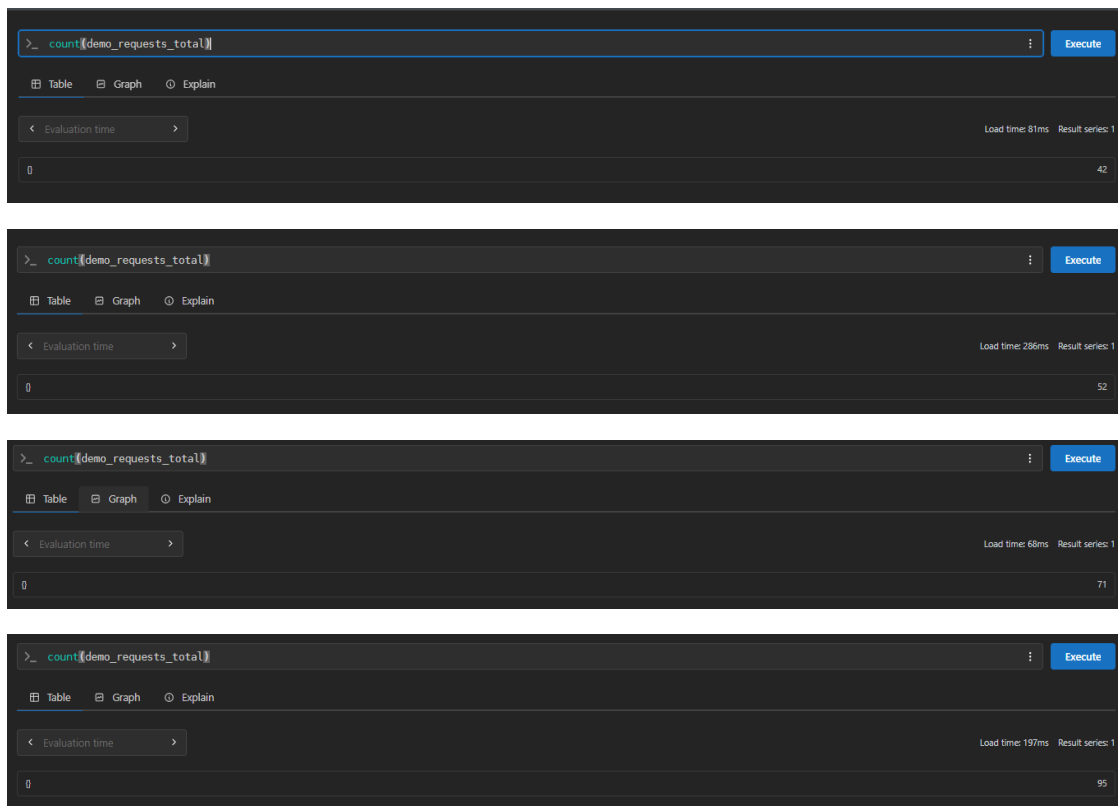


Figure: the series count climbing over time, 42, then 52, then 71, then 95, as more unique `request_id` values are created.

This confirms that one single counter, given a label with unique values, had actually created close to 100 separate time series inside Prometheus, one for every different id, instead of one series being updated 100 times.

Cleanup

A second, separate script was written using a fresh metric name and a fresh port, with the label removed entirely, to test the fix cleanly.

```
from prometheus_client import Counter, start_http_server
import time

demo_requests_v2 = Counter('demo_requests_v2_total', 'test counter without label')
```

```
start_http_server(8002)

for i in range(100):
    demo_requests_v2.inc()
    time.sleep(1)
```

Running `count(demo_requests_v2_total)` while this script ran showed a flat value of 1, no matter how long the script kept running or how many times it was checked. Removing the label completely stopped new series from being created.



Figure: the series count staying flat at 1, both times it was checked, once the label was removed.

Why This Matters at a Larger Scale

This test only reached 100 unique values on purpose, since the assignment specifically says not to try to push Prometheus further than that. At a real, production sized scale, for example a busy website handling thousands of requests every minute, giving a metric a label with a unique value per request would create thousands of brand new time series every single minute, permanently. This uses up memory and processing power very quickly, and can eventually make Prometheus unstable or unusable. The correct fix, shown by this project's own main metrics in Part B, is to only use labels that have a small, fixed set of possible values, like an HTTP method or a status code. Anything unique per request, such as a `request_id`, belongs in the logs instead, which is exactly why this project sends the `request_id` to Elasticsearch through Filebeat rather than attaching it as a Prometheus label.